MICROAPP BARBERÍA · BARBERYA

Base de datos

Hosting, costos y diseño del modelo relacional

Versión 1 · Noviembre 2026 · PostgreSQL

Resumen de decisiones

Tema	Decisión
Motor	**PostgreSQL** (relacional). Garantiza integridad entre turnos, clientes y plata, y se integra directo con Spring Boot + JPA.
Modelo de clientes	**Multi-tenant en una sola base**: cada barbería es un “tenant” y casi todas las tablas llevan `barberia_id`.
Barberos	Funciona igual para el **barbero independiente** (una barbería con un único miembro) y para la **barbería con varios barberos**.
Hosting sugerido	Desarrollo: **Neon Free** o **Supabase Free** (US$ 0). Lanzamiento: **Supabase Pro** (US$ 25/mes) o **Railway** junto con el backend (desde US$ 5/mes + uso).
Regla clave	La base de datos misma impide que un barbero tenga dos turnos superpuestos (exclusion constraint).

Contenido: 1. Dónde alojarla y cuánto cuesta · 2. Convenciones del modelo · 3. Tipos de relaciones · 4. Diagrama · 5. Tablas en detalle · 6. Reglas que garantiza la base · 7. Cómo encajan los dos tipos de cliente · 8. Próximos pasos · Anexo: script SQL completo.

1. Dónde alojar la base y cuánto cuesta

Para una app que arranca, conviene una **base gestionada** (managed): el proveedor se encarga de backups, actualizaciones y de que el servidor esté prendido. Ustedes solo usan la cadena de conexión desde Spring Boot. Todas las opciones de abajo son PostgreSQL estándar, así que se puede migrar de una a otra con `pg_dump` / `pg_restore` sin tocar el código.

Opciones comparadas

Proveedor	Plan gratis	Primer plan pago	Lo bueno	Ojo con
Supabase	US$ 0: 500 MB de base, 2 proyectos.	Pro: US$ 25/mes (8 GB de base).	Panel muy cómodo, backups, región São Paulo. Trae auth y storage (pueden ignorarlos y usar solo Postgres).	El free se pausa tras 7 días sin uso. En Pro, el cómputo extra se cobra aparte.
Neon	US$ 0: 0,5 GB y 100 horas de cómputo por proyecto.	Launch: por uso (~US$ 0,106 por hora de cómputo).	Se “apaga” cuando no hay tráfico y no cobra; branches de base para probar cambios.	Si la base queda prendida 24/7, la factura sube (puede pasar US$ 80/mes). Arranque en frío tras inactividad.
Railway	Prueba con créditos, sin plan gratis permanente.	Hobby: US$ 5/mes con US$ 5 de uso incluidos.	Hostea el backend Spring Boot y la base en el mismo lugar, deploy desde GitHub.	Se cobra RAM/CPU por uso; una base siempre prendida puede rondar US$ 15–20/mes.
Render	Postgres gratis de 1 GB que vence a los 30 días.	Basic-256MB ~US$ 6–7/mes; Basic-1GB ~US$ 19–20/mes.	Precio fijo y predecible; también hostea el backend.	El free no sirve para producción (se borra).
AWS RDS	Sin free tier para cuentas nuevas en 2026.	db.t4g.micro: ~US$ 12/mes en EE.UU., ~US$ 25/mes en São Paulo, + disco.	Lo más “profesional” y escalable; suma al CV.	Más configuración (VPC, seguridad, backups). Excesivo para el MVP.

Precios en dólares, sin impuestos, relevados en noviembre de 2026. Cambian seguido: revisarlos en la página de cada proveedor antes de contratar. Con tarjeta argentina se suman los impuestos sobre pagos en dólares.

¿Qué región elegir?

La región más cercana a Argentina en estos proveedores suele ser **São Paulo (sa-east-1)**. Lo importante es que el **backend y la base estén en la misma región**: cada pantalla hace varias consultas y si cada una cruza el continente se nota. Si el backend termina en EE.UU. (por ejemplo, en Railway o Render), la base tiene que estar ahí también.

Recomendación por etapa

Etapa	Qué usar	Costo estimado
Desarrollo (ahora)	Postgres local con Docker para cada uno + una base compartida en **Neon Free** o **Supabase Free** para probar juntos.	US$ 0
Beta con 1–5 barberías	Seguir en el free de Neon/Supabase o pasar a **Railway Hobby** con backend + base juntos.	US$ 0 – 20/mes
Lanzamiento con clientes que pagan	**Supabase Pro** (base) + backend en Railway/Render, o todo en Railway. Backups diarios activados.	US$ 25 – 50/mes
Crecimiento	Evaluar RDS o planes más grandes según uso real. Con este volumen de datos, una base chica aguanta cientos de barberías.	Según uso

Para dimensionar
Una barbería genera muy pocos datos: aunque tenga 1.000 turnos por mes, eso son kilobytes. Los 500 MB del plan gratis alcanzan para muchísimas barberías; lo que define cuándo pagar es necesitar que la base nunca se pause y tener backups confiables cuando haya clientes reales pagando.

2. Convenciones del modelo

¿Por qué relacional?

- Los datos están naturalmente conectados: un turno une cliente, barbero, servicio y horario. Las **foreign keys** garantizan que nunca exista un turno de un cliente borrado o de un servicio que no existe.
- Hay reglas "duras" de negocio (no superponer turnos, una sola suscripción activa) que Postgres puede hacer cumplir a nivel base, sin depender solo del código.
- Los reportes (ingresos del mes, gastos por categoría, clientes que más vienen) son `SUM`, `GROUP BY` y filtros por fecha: SQL puro.
- Encaja con lo que ya conocen: Spring Boot + Spring Data JPA + Flyway para migraciones.

Multi-tenant: una base para todas las barberías

Todas las barberías comparten las mismas tablas. Cada fila "pertenece" a una barbería mediante la columna `barberia_id`. Toda consulta del backend debe filtrar por la barbería del usuario logueado. Es el modelo más simple y barato de operar (una sola base, una sola migración). Las tablas que cuelgan de otra (por ejemplo `horarios_atencion`, que cuelga de un barbero) heredan la barbería a través de su padre.

Seguridad
Error típico a evitar: un endpoint como `GET /turnos/{id}` que busca solo por id. Siempre buscar por `id AND barberia_id`, si no, un usuario de una barbería podría ver datos de otra cambiando el número en la URL.

Tipos de datos elegidos

Uso	Tipo	Por qué
Identificadores	`BIGINT GENERATED ALWAYS AS IDENTITY`	Autoincremental estándar de Postgres, mapea a Long en Java. El link público usa el slug, no el id.
Plata	`NUMERIC(12,2)`	Exacto. Nunca usar FLOAT/DOUBLE para dinero (errores de redondeo). En Java: BigDecimal.
Fechas y horas de turnos	`TIMESTAMPTZ`	Guarda el instante exacto con zona horaria. Evita problemas de horario y servidores en otra zona.
Horario semanal	`TIME + SMALLINT (día)`	"Los martes de 10:00 a 19:00" no es un instante, es un patrón.
Fechas sin hora	`DATE`	Para gastos y vencimientos de suscripción.
Estados fijos	`ENUM`	Valores cerrados (estado del turno, rol, método de pago). En JPA: @Enumerated(EnumType.STRING).
Baja de registros	`activo BOOLEAN`	"Baja lógica": un servicio o barbero que se deja de usar se desactiva, no se borra, para no romper el historial de turnos.

3. Tipos de relaciones

Una relación se implementa con una **foreign key** (FK): una columna que apunta a la primary key (PK) de otra tabla. Según cómo se combine esa FK con restricciones, aparecen los distintos tipos:

Uno a muchos (1:N)

La más común. Una fila de la tabla A se relaciona con muchas de B, pero cada fila de B pertenece a una sola de A. Se implementa poniendo la FK en el lado "muchos".

En este modelo: Una barbería tiene muchos servicios; cada servicio es de una sola barbería → `servicios.barberia_id`. Lo mismo: barbería → clientes, barbero → turnos, cliente → turnos, suscripción → pagos.

En JPA: @ManyToOne en la entidad hija (Servicio) y, si hace falta, @OneToMany(mappedBy) en la padre.

Muchos a muchos (N:M)

Una fila de A se relaciona con muchas de B y viceversa. No se puede con una sola FK: se crea una **tabla puente** con dos FK.

En este modelo: Un barbero hace varios servicios y un servicio lo hacen varios barberos → `barbero_servicios` (PK compuesta: miembro + servicio). Usuarios ↔ barberías también es N:M: una persona puede trabajar en dos barberías → `miembros_barberia`, que además guarda datos propios de la relación (rol, si atiende turnos).

En JPA: Si la tabla puente solo tiene las dos FK: @ManyToMany + @JoinTable. Si tiene columnas propias (como el rol), conviene mapearla como entidad propia.

Uno a uno (1:1) / uno a cero-o-uno (1:0..1)

Cada fila de A se relaciona con, como máximo, una de B. Se implementa con una FK que además es **UNIQUE**.

En este modelo: Un turno puede tener como máximo un cobro → `cobros.turno_id` es UNIQUE. Si el turno todavía no se cobró, simplemente no hay fila. Y un cobro puede no tener turno (venta de un producto suelto), por eso la FK admite NULL.

En JPA: @OneToOne en la entidad Cobro con @JoinColumn(unique = true).

Relación opcional (FK nullable)

La FK puede quedar vacía (NULL). Indica que el vínculo no es obligatorio.

En este modelo: `clientes.usuario_id`: si el cliente reservó por la web tiene cuenta; si el barbero lo cargó a mano, no. `gastos.proveedor_id`: no todo gasto tiene proveedor (la luz, por ejemplo).

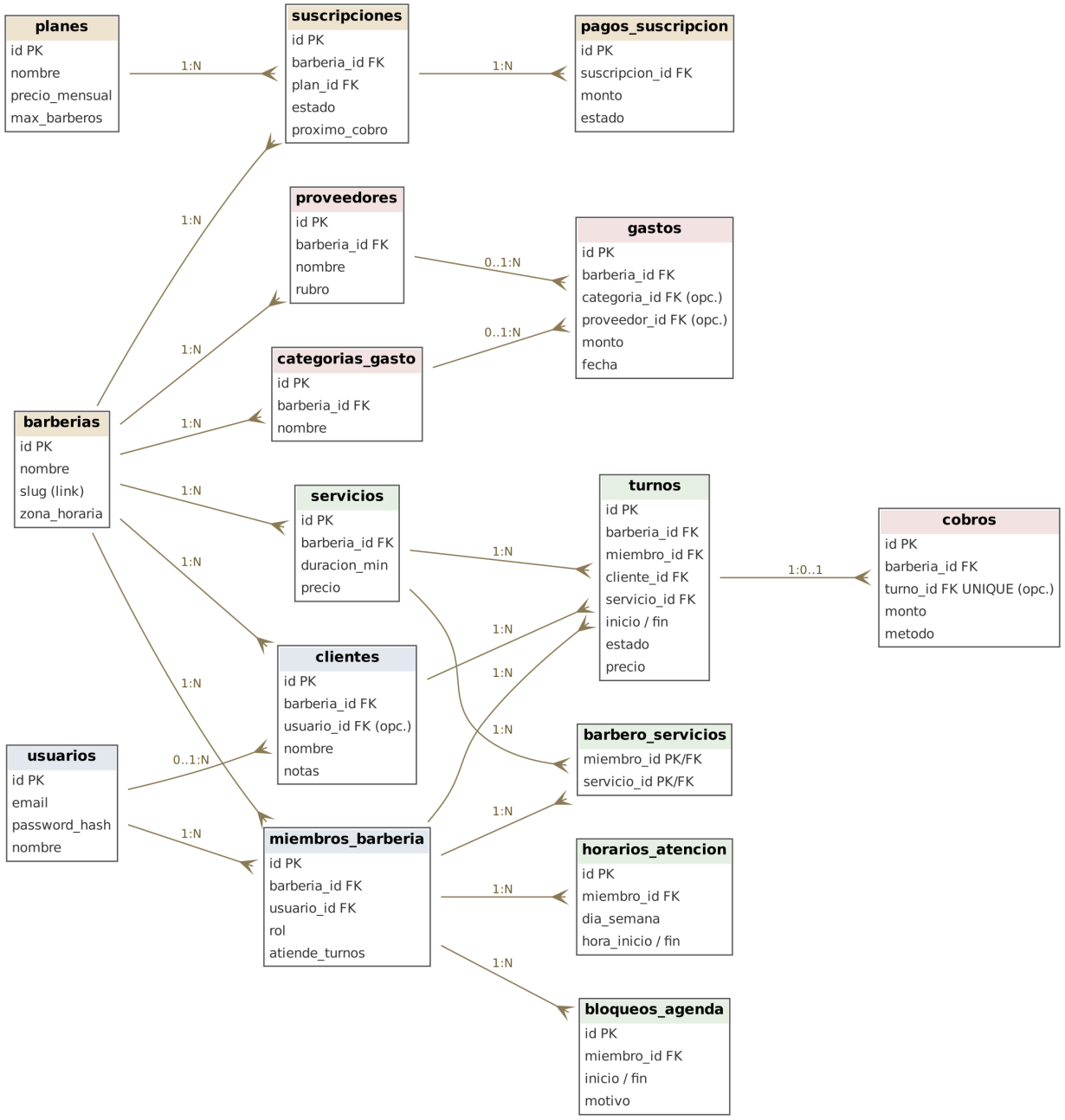
En JPA: @ManyToOne(optional = true) y la columna sin NOT NULL.

¿Qué pasa cuando se borra el padre? (ON DELETE)

Comportamiento	Qué hace	Dónde se usa
`CASCADE`	Borra también los hijos.	Todo lo que cuelga de barberias: si se elimina una barbería (baja definitiva), se va todo lo suyo.
`SET NULL`	Deja la FK del hijo en NULL.	gastos.categoria_id y gastos.proveedor_id: si se borra un proveedor, el gasto queda pero sin proveedor.
`RESTRICT (por defecto)`	Impide borrar el padre si tiene hijos.	turnos → clientes / servicios / barbero: no se puede borrar un cliente con turnos; se usa la baja lógica (activo = false).

4. Diagrama entidad-relación

Colores por módulo: **beige** plataforma y suscripción · **gris azulado** personas · **verde** agenda · **rosado** plata. La “pata de gallo” marca el lado “muchos”. Se muestran solo las columnas principales.



5. Tablas en detalle

16 tablas en cuatro módulos. En cada tabla: PK = primary key, FK = foreign key, NN = NOT NULL (obligatorio), UQ = único.

Módulo A · Plataforma y suscripción

Lo que ustedes le cobran a cada barbería.

planes

Catálogo de planes que ofrecen (por ejemplo “Independiente” para 1 barbero y “Barbería” para hasta 5).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
nombre	VARCHAR(50)	NN, UQ	Nombre comercial del plan.
precio_mensual	NUMERIC(12,2)	NN, ≥ 0	Precio en pesos.
max_barberos	INT	NN, > 0	Cuántos miembros que atienden turnos permite el plan. El backend lo valida al agregar un barbero.
activo	BOOLEAN	NN, def. true	Si se puede contratar hoy.

barberias

El “tenant”. Un barbero independiente también es una barbería (con un solo miembro).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
nombre	VARCHAR(120)	NN	Nombre que ve el cliente.
slug	VARCHAR(60)	NN, UQ	Parte del link público: barberya.com/b/*lo-de-tito*. Es como el cliente llega a la barbería.
direccion / telefono	VARCHAR		Datos de contacto.
logo_url	TEXT		URL de la imagen (el archivo vive en un storage, no en la base).
zona_horaria	VARCHAR(50)	NN	Por defecto America/Argentina/Buenos_Aires. Sirve para mostrar horarios correctamente.
activa	BOOLEAN	NN	Se pone en false si deja de pagar.
creado_en	TIMESTAMPTZ	NN, def. now()	Fecha de alta.

suscripciones

Qué plan tiene cada barbería y en qué estado está. Se guarda el historial: si cambia de plan, se cierra una fila y se abre otra.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
plan_id	BIGINT	FK → planes, NN	Plan contratado.
estado	ENUM	NN	PRUEBA, ACTIVA, VENCIDA, CANCELADA.
fecha_inicio / fecha_fin	DATE		Período de la suscripción.
proximo_cobro	DATE		Para avisar y bloquear si no paga.
mp_preapproval_id	VARCHAR(100)	UQ	Id de la suscripción en Mercado Pago, si cobran por ahí.

Índice único parcial ux_suscripcion_viva: una barbería no puede tener dos suscripciones en PRUEBA o ACTIVA al mismo tiempo.

pagos_suscripcion

Cada cobro mensual (aprobado o rechazado) de una suscripción.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
suscripcion_id	BIGINT	FK → suscripciones, NN	A qué suscripción corresponde.
monto	NUMERIC(12,2)	NN, ≥ 0	Importe cobrado.
pagado_en	TIMESTAMPTZ	NN	Cuándo se procesó.
estado	VARCHAR(20)	NN	APROBADO, RECHAZADO, PENDIENTE (lo que informa la pasarela).
referencia_externa	VARCHAR(100)	UQ	Id del pago en Mercado Pago. UQ evita registrar dos veces el mismo pago si llega el webhook repetido.

Módulo B · Personas

usuarios

Toda persona que inicia sesión: dueños, barberos y clientes. Una sola tabla de cuentas; lo que cada uno puede hacer depende de su relación con cada barbería.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
email	VARCHAR(255)	NN, UQ	Login. Único en toda la plataforma.
password_hash	VARCHAR(255)	NN	Hash BCrypt (Spring Security). Nunca la contraseña en texto.
nombre / apellido	VARCHAR(80)	nombre NN	Datos personales.
telefono	VARCHAR(30)		Para recordatorios por WhatsApp a futuro.
creado_en	TIMESTAMPTZ	NN	Fecha de registro.

miembros_barberia

Tabla puente usuarios ↔ barberías para el personal. Es la que hace que el modelo sirva para el independiente y para la barbería grande: el independiente tiene una fila (rol DUENO, atiende_turnos = true); una barbería con 4 barberos tiene 4 o 5 filas.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
usuario_id	BIGINT	FK → usuarios, NN	La persona.
rol	ENUM	NN	DUENO (todo, incluida la plata), BARBERO (su agenda), RECEPCION (agenda de todos, sin finanzas).
atiende_turnos	BOOLEAN	NN, def. true	Un dueño que no corta queda en false y no aparece para reservar.
nombre_visible / foto_url	VARCHAR / TEXT		Cómo lo ve el cliente al elegir barbero.
activo	BOOLEAN	NN	Baja lógica si deja de trabajar ahí.

UQ (barberia_id, usuario_id): una persona no puede estar dos veces en la misma barbería.

clientes

La ficha del cliente *dentro* de cada barbería. Si Juan va a dos barberías, tiene una sola cuenta en usuarios y una ficha en cada barbería, con notas independientes.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
usuario_id	BIGINT	FK → usuarios, opcional	NULL si el barbero lo cargó a mano (cliente que llamó por teléfono o vino sin turno).
nombre	VARCHAR(80)	NN	Cómo lo identifica la barbería.
telefono / email	VARCHAR		Contacto.
notas	TEXT		Lo que el barbero quiere recordar: “corte 2 al costado, no le gusta la navaja”.
creado_en	TIMESTAMPTZ	NN	Desde cuándo es cliente.

UQ (barberia_id, usuario_id): una cuenta tiene una sola ficha por barbería (los NULL no cuentan, así que se pueden cargar muchos clientes sin cuenta).

Módulo C · Agenda

servicios

Lo que ofrece la barbería: corte, barba, corte + barba, etc.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
nombre	VARCHAR(80)	NN	Nombre del servicio.
descripcion	TEXT		Detalle opcional.
duracion_min	INT	NN, > 0	Cuánto bloquea en la agenda. Con esto se calcula el fin del turno.
precio	NUMERIC(12,2)	NN, ≥ 0	Precio actual.
activo	BOOLEAN	NN	Desactivar en vez de borrar.

barbero_servicios

Tabla puente N:M: qué servicios hace cada barbero (por ejemplo, solo uno hace color).

Columna	Tipo	Restricciones	Descripción
miembro_id	BIGINT	PK, FK → miembros_barberia	El barbero.
servicio_id	BIGINT	PK, FK → servicios	El servicio.

horarios_atencion

Horario semanal de cada barbero. Permite turno partido: dos filas el mismo día (10–13 y 16–20).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
miembro_id	BIGINT	FK → miembros_barberia, NN	De quién es el horario.
dia_semana	SMALLINT	NN, 0 a 6	0 = domingo … 6 = sábado (igual que Postgres).
hora_inicio / hora_fin	TIME	NN, fin > inicio	Franja de atención.

bloqueos_agenda

Excepciones al horario: vacaciones, médico, feriado, un día que cierra temprano.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
miembro_id	BIGINT	FK → miembros_barberia, NN	Barbero bloqueado.
inicio / fin	TIMESTAMPTZ	NN, fin > inicio	Período sin turnos.
motivo	VARCHAR(120)		Texto libre, opcional.

turnos

La tabla central. Cada reserva une cliente, barbero, servicio y horario.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
miembro_id	BIGINT	FK → miembros_barberia, NN	Quién atiende.
cliente_id	BIGINT	FK → clientes, NN	A quién.
servicio_id	BIGINT	FK → servicios, NN	Qué se hace.
inicio / fin	TIMESTAMPTZ	NN, fin > inicio	fin = inicio + duración del servicio (lo calcula el backend).
estado	ENUM	NN, def. PENDIENTE	PENDIENTE, CONFIRMADO, COMPLETADO, CANCELADO, AUSENTE (no vino).
precio	NUMERIC(12,2)	NN	**Copia** del precio al reservar: si mañana el corte sube, los turnos viejos mantienen su precio.
notas	TEXT		Comentario del cliente o del barbero.
creado_en	TIMESTAMPTZ	NN	Cuándo se reservó.

Restricción `turnos_sin_superposicion`: impide que el mismo barbero tenga dos turnos que se pisen. Índices: `(barberia_id, inicio)` para la vista de agenda y `(cliente_id)` para el historial del cliente.

Módulo D · Plata

cobros

Ingresos de la barbería. Normalmente uno por turno completado, pero también ventas sueltas (productos).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
turno_id	BIGINT	FK → turnos, UQ, opcional	Relación 1:0..1 con turnos. NULL si es una venta sin turno.
monto	NUMERIC(12,2)	NN, ≥ 0	Lo que efectivamente se cobró (puede diferir del precio por propina o descuento).
metodo	ENUM	NN	EFECTIVO, TRANSFERENCIA, MERCADO_PAGO, TARJETA, OTRO.
cobrado_en	TIMESTAMPTZ	NN	Fecha del ingreso.
concepto	VARCHAR(120)		Para ventas sueltas: “cera para pelo”.

proveedores

Agenda de proveedores de la barbería.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
nombre	VARCHAR(120)	NN	Nombre o razón social.
rubro	VARCHAR(80)		Productos, insumos, mantenimiento…
telefono / email	VARCHAR		Contacto.
notas	TEXT		Condiciones, días de entrega.
activo	BOOLEAN	NN	Baja lógica.

categorias_gasto

Categorías que define cada barbería (alquiler, servicios, insumos, sueldos…). Al crear la barbería se pueden precargar unas por defecto.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
nombre	VARCHAR(60)	NN, UQ por barbería	Nombre de la categoría.

gastos

Egresos. Junto con cobros permite el resumen mensual: ingresos − gastos = ganancia.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK	Identificador autoincremental.
barberia_id	BIGINT	FK → barberias, NN	Barbería dueña de la fila (multi-tenant).
categoria_id	BIGINT	FK → categorias_gasto, opcional	ON DELETE SET NULL.
proveedor_id	BIGINT	FK → proveedores, opcional	ON DELETE SET NULL.
descripcion	VARCHAR(200)	NN	Qué se pagó.
monto	NUMERIC(12,2)	NN, > 0	Importe.
fecha	DATE	NN	Fecha del gasto.
metodo	ENUM		Cómo se pagó.
comprobante_url	TEXT		Foto de la factura (en un storage).
es_fijo	BOOLEAN	NN, def. false	Gastos fijos mensuales (alquiler, luz, internet) para proyectar el mes.

6. Reglas que garantiza la propia base

Validar en el backend está bien (da mejores mensajes de error), pero la base es la última barrera: aunque haya un bug o dos personas reserven el mismo horario en el mismo segundo, estas reglas no se pueden romper.

Regla	Cómo se implementa
Un barbero no tiene dos turnos superpuestos	EXCLUDE USING gist (miembro_id WITH =, tstzrange(inicio, fin) WITH &&) WHERE estado no es CANCELADO ni AUSENTE. Requiere la extensión btree_gist. Turnos contiguos (10:00–10:30 y 10:30–11:00) sí se permiten.
Una sola suscripción viva por barbería	Índice único parcial sobre barberia_id WHERE estado IN ('PRUEBA','ACTIVA').
Un turno se cobra una sola vez	cobros.turno_id es UNIQUE.
No hay montos negativos ni duraciones en cero	CHECK (monto >= 0), CHECK (duracion_min > 0).
Las franjas tienen sentido	CHECK (fin > inicio) en turnos, bloqueos y horarios.
Links y emails únicos	UNIQUE en barberias.slug y usuarios.email.
No se duplica gente en una barbería	UNIQUE (barberia_id, usuario_id) en miembros_barberia y clientes.
No queda nada huérfano	Foreign keys en todas las relaciones.

Verificado
Este script se probó en PostgreSQL: crea las 16 tablas, acepta turnos contiguos y rechaza un turno superpuesto y una segunda suscripción activa para la misma barbería.

En Spring Boot, el intento de insertar un turno superpuesto llega como DataIntegrityViolationException: hay que atraparla y devolver un 409 Conflict con un mensaje tipo “ese horario ya no está disponible”.

7. Cómo encajan los dos tipos de cliente

	Barbero independiente	Barbería con varios barberos
barberias	1 fila	1 fila
miembros_barberia	1 fila: rol DUENO, atiende_turnos = true	Dueño (DUENO, puede no atender) + una fila por barbero (BARBERO) + recepcionista opcional
Plan	“Independiente” (max_barberos = 1)	“Barbería” (max_barberos = 5, por ejemplo)
Reserva del cliente	Elige servicio y horario; el barbero se asigna solo	Elige servicio, barbero (o “cualquiera”) y horario
Pantallas	Ve todo: agenda, clientes, plata	BARBERO ve su agenda; DUENO ve todo y las finanzas

El cambio de independiente a barbería con equipo no requiere tocar la base: se cambia el plan y se agregan filas en miembros_barberia.

Flujo de una reserva

- El cliente entra por el link (`/b/{slug}`) → el backend obtiene la barbería por slug.
- Se registra o inicia sesión → se crea en `usuarios` y, si es su primera vez en esa barbería, una ficha en `clientes`.
- Elige servicio → se filtran los barberos que lo hacen (`barbero_servicios`).

- El backend calcula los horarios libres: horarios_atencion − bloqueos_agenda − turnos existentes, en bloques de la duración del servicio.
- Confirma → INSERT en `turnos`. Si otro le ganó ese horario, la base lo rechaza y se le ofrece otro.
- Después del corte, el barbero marca COMPLETADO y registra el cobro en `cobros`.

8. Próximos pasos

- **Migraciones con Flyway**: poner el script del anexo como `V1__esquema_inicial.sql` en `src/main/resources/db/migration`. Cada cambio futuro es un archivo nuevo (V2, V3...), nunca editar uno ya aplicado. Configurar `spring.jpa.hibernate.ddl-auto=validate` para que Hibernate no modifique la base.
- **Docker Compose** con `postgres:16` (o la versión que use el proveedor) para que los dos desarrollen con la misma base local.
- **Datos semilla**: los planes y las categorías de gasto por defecto en una migración aparte.
- **Filtro por barbería** centralizado en el backend (por ejemplo, el barberia_id sale del token JWT del usuario logueado, nunca de un parámetro que manda el front).
- **Más adelante**: turnos con varios servicios (tabla turno_servicios), recordatorios por WhatsApp (tabla de notificaciones), stock de productos.

Fuentes de precios

• Supabase: supabase.com/pricing (resumido en jetadmin.io y focusreactive.com, 2026).

• Neon: neon.com/pricing (resumido en budgetforge.dev y jetadmin.io, 2026).

• Railway: railway.com/pricing (resumido en livemy.app y frontdeskreview.com, 2026).

• Render: render.com/pricing y docs de Render Postgres (resumido en pricingsaas.com y kuberns.com, 2026).

• AWS RDS: aws.amazon.com/rds/postgresql/pricing (por región en bytebase.com/dbcost; free tier en infratally.com, 2026).

Anexo · Script SQL completo (PostgreSQL)

Listo para usar como primera migración de Flyway.

```
-- Microapp Barbería — esquema PostgreSQL (v1)
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ===== Tipos enumerados =====
CREATE TYPE rol_miembro        AS ENUM ('DUENO', 'BARBERO', 'RECEPCION');
CREATE TYPE estado_suscripcion AS ENUM ('PRUEBA', 'ACTIVA', 'VENCIDA', 'CANCELADA');
CREATE TYPE estado_turno       AS ENUM ('PENDIENTE', 'CONFIRMADO', 'COMPLETADO', 'CANCELADO', 'AUSENTE');
CREATE TYPE metodo_pago        AS ENUM ('EFECTIVO', 'TRANSFERENCIA', 'MERCADO_PAGO', 'TARJETA', 'OTRO');

-- ===== Plataforma (lo que cobran ustedes) =====
CREATE TABLE planes (
  id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre          VARCHAR(50)    NOT NULL UNIQUE,
  precio_mensual  NUMERIC(12,2)  NOT NULL CHECK (precio_mensual >= 0),
  max_barberos    INT            NOT NULL CHECK (max_barberos > 0),
  activo          BOOLEAN        NOT NULL DEFAULT TRUE
);

CREATE TABLE barberias (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre       VARCHAR(120) NOT NULL,
  slug         VARCHAR(60)  NOT NULL UNIQUE,      -- barberya.com/b/{slug}
  direccion    VARCHAR(200),
  telefono     VARCHAR(30),
  logo_url     TEXT,
  zona_horaria VARCHAR(50)  NOT NULL DEFAULT 'America/Argentina/Buenos_Aires',
  activa       BOOLEAN      NOT NULL DEFAULT TRUE,
  creado_en    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE suscripciones (
  id                 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id        BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  plan_id            BIGINT NOT NULL REFERENCES planes(id),
  estado             estado_suscripcion NOT NULL DEFAULT 'PRUEBA',
  fecha_inicio       DATE   NOT NULL DEFAULT CURRENT_DATE,
  proximo_cobro      DATE,
  fecha_fin          DATE,
  mp_preapproval_id  VARCHAR(100) UNIQUE,         -- id de la suscripción en Mercado Pago
  creado_en          TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Una sola suscripción "viva" por barbería
CREATE UNIQUE INDEX ux_suscripcion_viva ON suscripciones (barberia_id)
  WHERE estado IN ('PRUEBA', 'ACTIVA');

CREATE TABLE pagos_suscripcion (
  id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  suscripcion_id      BIGINT NOT NULL REFERENCES suscripciones(id) ON DELETE CASCADE,
  monto               NUMERIC(12,2) NOT NULL CHECK (monto >= 0),
  pagado_en           TIMESTAMPTZ NOT NULL DEFAULT now(),
  estado              VARCHAR(20) NOT NULL,       -- APROBADO / RECHAZADO / PENDIENTE
  referencia_externa  VARCHAR(100) UNIQUE         -- id de pago de Mercado Pago
);

-- ===== Personas =====
CREATE TABLE usuarios (
  id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email          VARCHAR(255) NOT NULL UNIQUE,
  password_hash  VARCHAR(255) NOT NULL,           -- BCrypt, nunca la contraseña
  nombre         VARCHAR(80)  NOT NULL,
  apellido       VARCHAR(80),
  telefono       VARCHAR(30),
  creado_en      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- Tabla puente usuarios <-> barberias (N:M) con rol
CREATE TABLE miembros_barberia (
  id               BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id      BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  usuario_id       BIGINT NOT NULL REFERENCES usuarios(id),
  rol              rol_miembro NOT NULL,
  atiende_turnos   BOOLEAN NOT NULL DEFAULT TRUE,
  nombre_visible   VARCHAR(80),
  foto_url         TEXT,
```

```
  activo           BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (barberia_id, usuario_id)
);

-- Ficha del cliente DENTRO de cada barbería
CREATE TABLE clientes (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id  BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  usuario_id   BIGINT REFERENCES usuarios(id),    -- NULL = cliente cargado a mano, sin cuenta
  nombre       VARCHAR(80)  NOT NULL,
  telefono     VARCHAR(30),
  email        VARCHAR(255),
  notas        TEXT,                              -- "corte 2 al costado, no le gusta la navaja"
  creado_en    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  UNIQUE (barberia_id, usuario_id)
);

-- ===== Agenda =====
CREATE TABLE servicios (
  id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id   BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  nombre        VARCHAR(80)   NOT NULL,
  descripcion   TEXT,
  duracion_min  INT           NOT NULL CHECK (duracion_min > 0),
  precio        NUMERIC(12,2) NOT NULL CHECK (precio >= 0),
  activo        BOOLEAN       NOT NULL DEFAULT TRUE
);

-- Qué servicios hace cada barbero (N:M)
CREATE TABLE barbero_servicios (
  miembro_id   BIGINT NOT NULL REFERENCES miembros_barberia(id) ON DELETE CASCADE,
  servicio_id  BIGINT NOT NULL REFERENCES servicios(id) ON DELETE CASCADE,
  PRIMARY KEY (miembro_id, servicio_id)
);

CREATE TABLE horarios_atencion (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  miembro_id   BIGINT   NOT NULL REFERENCES miembros_barberia(id) ON DELETE CASCADE,
  dia_semana   SMALLINT NOT NULL CHECK (dia_semana BETWEEN 0 AND 6),  -- 0 = domingo
  hora_inicio  TIME     NOT NULL,
  hora_fin     TIME     NOT NULL,
  CHECK (hora_fin > hora_inicio)
);

CREATE TABLE bloqueos_agenda (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  miembro_id  BIGINT      NOT NULL REFERENCES miembros_barberia(id) ON DELETE CASCADE,
  inicio      TIMESTAMPTZ NOT NULL,
  fin         TIMESTAMPTZ NOT NULL,
  motivo      VARCHAR(120),
  CHECK (fin > inicio)
);

CREATE TABLE turnos (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id  BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  miembro_id   BIGINT NOT NULL REFERENCES miembros_barberia(id),
  cliente_id   BIGINT NOT NULL REFERENCES clientes(id),
  servicio_id  BIGINT NOT NULL REFERENCES servicios(id),
  inicio       TIMESTAMPTZ NOT NULL,
  fin          TIMESTAMPTZ NOT NULL,
  estado       estado_turno  NOT NULL DEFAULT 'PENDIENTE',
  precio       NUMERIC(12,2) NOT NULL,            -- copia del precio al momento de reservar
  notas        TEXT,
  creado_en    TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (fin > inicio),
  -- Un barbero no puede tener dos turnos que se pisen (salvo cancelados/ausentes)
  CONSTRAINT turnos_sin_superposicion EXCLUDE USING gist (
    miembro_id WITH =,
    tstzrange(inicio, fin, '[)') WITH &&
  ) WHERE (estado NOT IN ('CANCELADO', 'AUSENTE'))
);
CREATE INDEX ix_turnos_barberia_inicio ON turnos (barberia_id, inicio);
CREATE INDEX ix_turnos_cliente ON turnos (cliente_id);

-- ===== Plata =====
CREATE TABLE cobros (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id  BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  turno_id     BIGINT UNIQUE REFERENCES turnos(id),  -- UNIQUE => 1:1 opcional con turnos
```

```
  monto        NUMERIC(12,2) NOT NULL CHECK (monto >= 0),
  metodo       metodo_pago   NOT NULL,
  cobrado_en   TIMESTAMPTZ   NOT NULL DEFAULT now(),
  concepto     VARCHAR(120)                          -- p. ej. venta de cera sin turno
);
CREATE INDEX ix_cobros_barberia_fecha ON cobros (barberia_id, cobrado_en);

CREATE TABLE proveedores (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id  BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  nombre       VARCHAR(120) NOT NULL,
  rubro        VARCHAR(80),
  telefono     VARCHAR(30),
  email        VARCHAR(255),
  notas        TEXT,
  activo       BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE categorias_gasto (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id  BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  nombre       VARCHAR(60) NOT NULL,
  UNIQUE (barberia_id, nombre)
);

CREATE TABLE gastos (
  id               BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  barberia_id      BIGINT NOT NULL REFERENCES barberias(id) ON DELETE CASCADE,
  categoria_id     BIGINT REFERENCES categorias_gasto(id) ON DELETE SET NULL,
  proveedor_id     BIGINT REFERENCES proveedores(id)      ON DELETE SET NULL,
  descripcion      VARCHAR(200)  NOT NULL,
  monto            NUMERIC(12,2) NOT NULL CHECK (monto > 0),
  fecha            DATE          NOT NULL DEFAULT CURRENT_DATE,
  metodo           metodo_pago,
  comprobante_url  TEXT,
  es_fijo          BOOLEAN NOT NULL DEFAULT FALSE,  -- alquiler, luz, internet...
  creado_en        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_gastos_barberia_fecha ON gastos (barberia_id, fecha);
```